



Version 1.03.0

Douglas Harvey-Marose - Copyright 2025

Quick Start

Introduction

UCurses is a recreation of DOS VGA terminal control libraries such as Curses within Unity. This UI framework allows you to create accurate recreations of DOS era Text User Interface (TUI) programs.

Dependencies

UCurses requires installation of the Unity Input System through the Package Manager.

Setup

To setup and deploy UCurses, from the main Unity menu select the [Window → UCurses Setup](#). This will open the setup window.

Setup Window

The setup window allows you to set the options for how UCurses is deployed.

- **Grid Size:** This setting controls how many row and columns the character grid on screen has, and the base rendering resolution that the grid will use. The dropdown has a selection of curated VGA text modes that allow a range of different looks for your grid. If you would instead like to create your own selection choose the Custom option. This adds additional options.
 - **Size of Grid:** Sets how many row and columns are in the character grid.
 - **Dos Screen Resolution:** Sets the reference resolution that the grid will use.
 - **Character Size:** The size of each character space in the grid.
 - **Offset Line:** Adds a extra line after each character space.
- **Aspect Ratio:** This sets the aspect ratio for the display output to the main camera. The dropdown includes a selection of commonly used aspect ratios, and Original which keeps the aspect ratio of the set Dos resolution.
- **Screen Alignment:** This sets the horizontal alignment of the display output window. There are three options: Middle, Left, and Right.
- **Alignment Offset:** This sets an offset for the horizontal screen position in pixels. Using a positive value moves the window to the right, and negative value to the left.
- **Aspect Ratio:** This sets the aspect ratio for the display output to the main camera.
- **Character Filter:** This controls the texture filtering of the character sprites used in the grid.
- **Screen Filter:** This controls the texture filtering of the screen display.

- **Set Grid Button:** Clicking the Set Grid Button for the first time deploys UCurses to the current scene, creating all needed objects in the scene as children of an object called UCurses. Once the deploy of UCurses is finished, it is ready to be used in any script within the scene. Once UCurses is deployed for the first time clicking the button again will set the UCurses settings to the current selection of settings in the Setup Window.

Warning: It is highly recommended that you create a new scene for use with UCurses. UCurses removes all Camera and Canvas objects when deployed and replaces them with its own. If you do wish to deploy to a scene that is using any of these objects in a custom way, you may have unpredictable results. If you do deploy to a scene already in development, it is highly recommended that you backup the scene beforehand.

Basic Usage

UCurses is a coding centric framework and all of its core functionalities are used through scripting. The UCurses framework script is a singleton class that can be accessed from any script in a scene that UCurses is deployed. To access UCurses in a script, add `using UCursesInclude` at the start of your script.

The static singleton reference to the UCurses class can be found within `UCurses.Instance`, from there you can find all UCurses public properties and methods.

Example: `UCurses.Instance.printCharacter(new Vector2Int(0,0), 'A', Color.red);`

Core Methods

The following methods are used to manipulate the character grid, and should be all you need to start programming basic software with Ucourses. Full detailed and expanded information on these methods and on the rest of the framework can be found in the API Documentation file.

void printCharacter(Vector2Int position, char character, Color32 color)
Prints a character with a given color on the character grid.
void printCharactersRectangle(Vector2Int position, char character, Color32 color, int width, int height)
Prints a rectangle of a character with a given color on the character grid.
void printString(Vector2Int position, string charString, Color32 color)
Prints a string of a characters with a given color on the character grid.
void printNumberInt(Vector2Int position, int numericValue, Color32 color)
Prints a whole number with a given color on the character grid.
void printNumberDouble(Vector2Int position, double numericValue, Color32 color)
Prints a decimal number with a given color on the character grid.
void printBackground(Vector2Int position, Color32 color)
Changes the background color of a character grid space.
void printBackgroundRectangle(Vector2Int position, Color32 color, int width, int height)
Changes the background color of a rectangle of character grid spaces.
Vector2Int mousePosition()
Retrieves the position of the mouse on the character grid. Returns (-1, -1) if outside of the character grid.
void characterBlink(Vector2Int position, bool enable)
Sets a character on the character grid to blink.
void characterUnderlined(Vector2Int position, bool enable)
Sets a character on the character grid to be underlined.
char asciiToChar(int asciiValue)
Returns a ascii value from the charSprites array by its char value. -1 is error.